Semantic and Textual Inference Chatbot Interface (STICI-Note) - Part 1: Planning and Prototyping

The start of my RAG to riches story

STICI-note

Published: Mon, 27 May 2024

Last modified: Tue, 04 Jun 2024

Introduction

In this three-part series, I will be talking you through how I built the Semantic and Textual Inference Chatbot Interface (or STICI-note for short), a locally run RAG chatbot that uses unstructured text documents to enhance its responses. I came up with the name when I was discussing this project with a friend and asked him whether he had any ideas of what to call it. He said, "You should call it sticky because it sounds funny." The name... stuck...

The code for this project is available here.

In this part, I will be planning the project from the tech stack to the techniques I will use, and I will be building a prototype. I will be discussing all of the choices I made and all of the choices I didn't make, so I hope you find this insightful. Without further ado, let's get started.

The Problem

In my spare time, I occasionally play Dungeons and Dragons (DnD), a tabletop roleplaying game, and the stories are often told over several months, so details can be easily lost or forgotten over time. I can write notes on my laptop, but sometimes regular text search does not always provide me with the results I want when trying to search for specific notes. Some common examples include when a keyword is used often (e.g., I might write a lot about the actions of "Miles Lawson," but only one segment of text might describe who he is, making searching for information on his character

like finding a needle in a haystack) or when I simply cannot think of the correct keyword to search (e.g., what if I search “silver” instead of “gold”?).

One day, I thought to myself that it’d be great if I had a tool that I could write my DnD notes into in an unstructured way and retrieve the information at any time with simple questions like “Who is Miles Lawson?” or “How much silver did I pay for a stay in ye olde tavern?”. This tool could be extended to be used for querying my notes on many things that are not available online (and therefore not searchable on a search engine), such as documentation on software that I build, notes on things that I’m learning about, such as AWS cloud infrastructure, and my diary of my deepest thoughts and feelings (at least I hope this is not available online). And thus, I decided to start working on STICI-note because the tools available online that do this cost money and run on the cloud, and I’m a tight-fisted guy who’s very sceptical about company promises to not sell your data.

Narrowing Down Features

As with all projects, I began by deciding what features I needed from this tool.

Required features:

Chatbot that you can ask questions and get answers in response (conversational memory is not required).
Information is taken from an unstructured text file.
It must be able to tell me if it doesn’t know the answer to my question.
Fast.
Efficient enough to run on my MacBook with other programs without any performance issues.
Locally run for privacy and to ensure it will always be free, runnable, and consistent.
Conversational memory is the memory of previous interactions given to an LLM. I decided not to require it as a feature because I just need the AI to answer my questions about the given text. It might be added as a feature in the future if I feel like I need it, but I do not plan to include it in the initial version of STICI-note.

I knew that limiting it to running on my M1 MacBook with 8 GB of memory would greatly limit the performance of the tool as I would not be able to access truly large language models like GPT-4 and Claude 3 Opus, but I decided to do it anyway primarily for privacy but also to remove dependencies on external organisations to reduce the maintenance required for the tool in the future.

Planning How to Evaluate and Compare Solutions

If you don't evaluate a solution, how do you know whether it's an effective solution? You don't.

I next planned how I would evaluate different variations of the tool. While I do not evaluate anything in this part, I decided to sketch out a rough plan of how I would evaluate different solutions to encourage designing a testable AI in the same way that Test-Driven Development (TDD) encourages you to write testable code.

At first, I considered using Wikipedia pages as the data source and making my own questions about the content of the pages before I realised that this would lead to data leakage as many LLMs are trained on Wikipedia data.

An alternative dataset that I considered using for evaluation is the TREC 2024 RAG Corpus. This is a 28 GB corpus of text documents scraped from the internet. This corpus comes with thousands of queries, and the relevant documents for each query have been tagged as such. This is an amazing corpus for training and evaluating vector DataBase (DBs). Ignoring the fact that its questions do not come with answers, meaning I would have to write my own answers to use the document, there is one glaring flaw that makes it unusable for my use case: the documents are generally relatively short and describe a large variety of things. In my use case, I expect documents to be long and typically written about the same topic. If I were to use the corpus, I would have to stick documents together to present a realistic challenge in the semantic search of the vector DB vector space, but as each document will likely be about very different topics (e.g., one might be about aviation while another might be about economics), context retrieval would be unrealistically easy.

Another alternative evaluation dataset that I considered using was a synthetic dataset. By following a method like this, I can use an LLM to generate synthetic context and questions automatically. I decided not to do this as I was concerned that this would produce bad-quality data with a massive bias towards things an LLM might already know, despite the use case expecting data that the LLM does not already know.

Because the documents in my evaluation corpus need to be thousands of words in length while staying relevant to a topic and they need to include information that will not be in the LLM's training data, I decided that it would be best to manually curate a small dataset to evaluate my models. I plan to create documents from sources on the internet like videogame wikis, people's blogs, and scientific journals and write my own pairs of questions and answers about them. I will then evaluate the difference between the model's answer and my answer using a semantic similarity score.

RAG vs a Really Big Context Window vs Infinite Context

To be able to answer questions about documents, the LLM would need to have access to information from the documents. I thought of three potential solutions for this:

Retrieval Augmented Generation (RAG)
An LLM with a really big context window
An LLM that supports infinite context length
Using an LLM with a really big context window such as Anthropic's Claude 3 and Google's Gemini 1.5 would certainly give me the best results as it would allow inference using completely unfiltered and uncompressed context as they can handle inputs of over 700,000 words, but these models are closed-source, and there is absolutely no chance of a model of this size fitting into my tiny M1 MacBook with 8 GB of memory.

By "an LLM that supports infinite context length," I mean models like Mamba, Megalodon, and Infini-attention that compress context into a finite space. I decided not to use a model like this for two main reasons. Firstly, I have concerns about the performance. These architectures are in their infancy, and I do not expect them to outperform equivalently-sized traditional transformers. Secondly, as these architectures are very new and experimental, I do not expect much support for them, especially for Apple's

M-series of chips, which have their own graphics API, metal, that is required for GPU acceleration on my MacBook. These architectures are very interesting, and I would love to try them out, but for this project, I will have to settle for a more tried-and-tested approach.

The more tried and tested approach that I settled with is RAG. It is an incredibly popular technique for allowing LLMs to make use of information that is too big to fit in their context windows. This technique is known to perform very well, is incredibly well supported by LLM frameworks like LangChain and llamaindex, and works well in resource-constrained environments like on my laptop. Given all this, RAG was an obvious choice.

Optimising Models for Limited Memory Environments

Next, I decided to investigate what kinds of optimisation strategies were available to use to try to fit bigger models into my M1 chip, as bigger LLMs typically perform better (I know, a groundbreaking revelation). To optimise the LLMs that I use, I considered four different techniques:

Quantisation
Model pruning
Model/knowledge distillation
AirLLM
Quantisation is the most common method for making ML models smaller (and therefore faster and more capable of fitting into smaller spaces). It's well known for improving speed and memory usage with little loss in accuracy in return, which makes it very popular for production-level AI. Quantising a model would require being able to fit it into your GPU, but I'm trying to quantise a model so that it can fit into my GPU, so without additional computing power, it's a bit of a chicken and egg problem. Luckily, because this is such a popular technique, there are many quantised versions of large, high-performance LLMs available on HuggingFace that I can use, so there is no need to do this myself.

Model pruning is a less common method for reducing model sizes, but it is not a technique that one should overlook. This is a technique that can be combined with quantisation (or used on its own) to further reduce models at the expense of accuracy, but I do not plan to apply it myself due to its complexity and the fact that quantisation has the same effect. There are

pruned models available on HuggingFace, but they don't typically perform as well as equivalently sized quantised models, so I do not plan to use any unless they have particularly good evaluation results on a common LLM benchmark.

Model/knowledge distillation is another size reduction technique that I considered. Unlike the previous techniques, model distillation can actually improve accuracy in domain-specific tasks while making a smaller model. As with quantisation and model pruning, I will use pre-distiled models, but I will not distil any models myself due to the computing power it requires (which admittedly is far less than training a model from scratch) and the complexity it would add to the project.

The final optimisation technique that I considered, AirLLM, is quite different from the others in that instead of optimising the model weights, it optimises the model inference. Typically, LLMs are loaded onto the GPU in their entirety, requiring a lot of VRAM to run the larger, better-performing models. AirLLM is an open-source library that tackles this problem by using layered inference, an inference technique that involves loading layers individually when they are needed instead of all at once. This allows larger models to fit into smaller memory spaces without degrading performance. This method definitely has a high potential for accuracy, but I decided not to use it as I am concerned about compatibility and reliability issues as it is a new tool and the GitHub repo has been developed by a single person, so support for it is likely to be limited. Additionally, my M1 chip only has 8 GB of memory shared between the CPU and GPU, which is excellent for reducing data loading overhead costs, but it means that larger models that require AirLLM will be loaded directly from the SSD, so I am concerned that the model layer loading and unloading will become a massive bottleneck when doing inference on larger models. I will reconsider this option if I find that the models that can run on my MacBook do not have satisfactory accuracy.

What Models Even Run on My MacBook?

After getting an idea of what kinds of optimisation techniques were available, I decided to conduct some tests to find out what LLMs would actually run on my MacBook. You could argue that since I am only building a prototype right now, I only need to find one LLM that performs well on my

MacBook, but I decided to find five models instead to give me an idea of what kinds of models I will be able to use. In particular, I wanted to know how big the models I could run were and what precision the weights would likely be.

I tested models that I had heard were good or showed decent results on the Hugging Face H4 Open LLM Leaderboard. I found LM Studio incredibly useful for testing out LLMs without having to write any code, which saved me a lot of time. Below are the five suitable models that I found that could run on my MacBook and were fast enough to satisfy me:

tinyllama-1.1b-chat-v1.0 Q6_K
Phi 3 Q4_K_M
bartowski/dolphin-2.8-experiment26-7b-GGUF Q3_K_L
mgonzs13/Mistroll-7B-v2.2-GGU
QuantFactory/Meta-Llama-3-8B-Instruct Q3_K_M
These models range from 1.17 GB up to 4.02 GB in size. I chose not to use any models that were any larger than 4 GB, as with only 8 GB of memory available, I expect that models that are any bigger would seriously impact the other applications that the user (i.e., me) is running on their device.

I will likely test out more models than this while testing out different configurations for the tool, but for the prototype, this is enough.

A Model Without a Framework is Like a Car Engine Without a Chassis

To run my models, I could have written a framework for loading, unloading, and executing the models, passing context and queries to the models, and integrating the vector DB (more on that later) with the inference model from scratch, but I didn't because I'm not insane and I am not trying to learn how to make ML frameworks. A lot of university students (myself included) are conditioned to try to build things from scratch for fear of plagiarism and because they are used to building things from scratch as a learning exercise (a very effective one in my opinion), so it's difficult to unlearn the DIY mindset, but it's simply a lot quicker and a lot more reliable to use libraries than to reinvent the wheel. Saying that, I decided to use a relatively simple tech stack.

Python was an obvious choice for me, given that I have a lot of experience with it and that it has an abundance of support for machine learning applications. I decided to use LangChain to orchestrate my RAG process from the vector DB to the inference, as it is a flexible tool for composing NLP pipelines. It is very popular and reliable, and it includes a lot of tools that make developing NLP applications easier. I considered using LlamaIndex as it is built more specifically for RAG applications, but LangChain is more general-purpose, which I expect will make it more extensible for times when I might want to add more features in the future. Additionally, I am more likely to use LangChain again for other applications in the future, so the experience will be more useful. I also considered using LitGPT, but I had some issues getting it to work with the M1 chip's Metal Performance Shaders (MPS), so I decided not to use it for fear of incompatibility. LitGPT is also intended more for training and fine-tuning LLMs, so it is likely not the best tool for simply deploying them in an application.

To run inference on my models, I will need another library to actually execute the model. As I am using a range of pre-trained models, I will mainly use HuggingFace's transformers library and the Python bindings for llama.cpp library to load and execute models, as these provide simple interfaces for inference, and I don't need the additional control that deep learning frameworks like TensorFlow, PyTorch, and JAX provide as I'm using pre-trained models. As I mentioned earlier, AirLLM is still on the table if I need better performance, but I will find out while evaluating models whether this is necessary.

Magnitude + Direction DBs

Since I was using RAG, I needed a vector DB. Deciding which one to use was the final step of the research and also the most difficult one, as vector DBs are the technology that I am least familiar with. For the vector DB, my main requirements were simple: it needed to be lightweight, locally runnable on a laptop, fast, and compatible with my MacBook. Lightweight and locally runnable sound like similar things, but I mean different things by each phrase. The locally runnable one is quite self-explanatory, but by lightweight, I mean quite minimal computation requirements that don't add features like heavy amounts of redundancy and heavy caching, which are

useful for large-scale systems, but will simply drain resources in my application that is designed to run alongside an LLM and the user's other applications on 8 GB of memory shared between the CPU and GPU.

I considered sixteen different vector DBs, but there were three different solutions that stood out to me for my use case: Chroma, Qdrant, and Vespa. These were all lightweight vector DBs that fit all of my requirements, but I decided to pick Chroma out of the three of them because it has a very batteries included approach, which is very useful for getting an application working quickly.

Below is an overview of my planned tech stack. I did not include any UI, as there are many options for this, and I plan to decide on the UI when building the final product.

The tech stack and data flow that I plan to use for STICI-note.
The tech stack and data flow that I plan to use for STICI-note.

Making the Prototype (A.K.A. the Easy Part)

The final step was to actually build the prototype/proof of concept (PoC) to find out whether this idea can actually be implemented using the tech stack that I have picked. I decided to implement it using the smallest of the 5 models that I had found earlier, tinyllama-1.1b-chat-v1.0 Q6_K, to minimise the chances of having any memory issues.

You can find my prototype code here.

For the prototype, I decided to make very basic configurations for everything to develop it quickly, and I tested it by asking a question about Grace Hopper, one of the most influential programmers to have ever lived, in two versions of the prototype: one with Grace Hopper's Wikipedia page in its vector DB and one with Alan Turing's Wikipedia page in its vector DB.

The question was:

"What programming language was Grace Hopper known for creating?"

As expected, the version with Alan Turing's Wikipedia page failed miserably, hallucinating its entire response:

Grace Hopper was a computer scientist who is best known for her work on developing an early version of BASIC, which was one of the first low-level programming languages designed for mainframe computers. Hopper worked at Digital Equipment Corporation (DEC) in the mid-1960s and was responsible for creating several key BASIC versions that were used by DEC's mainframe computers. In 1973, she left DEC to work on a new low-level programming language called PASCAL, which became one of the most popular programming languages of its time. However, her contributions to BASIC are still remembered today as it was one of the first widely used and easy-to-understand programming languages for mainframe computers.

This behaviour of hallucinating will need to be fixed in the final product, likely through prompt engineering.

As expected, the version with Grace Hopper's Wikipedia page gave a much more accurate response but it unexpectedly contained far more spelling errors:

Grace Hopper is credited with creating the first computer programming languaige called FLOW-MATIC. This language was developed in 1954 and was based on English words. Hopper's work on this languaige led to a machine-indepenent programming language, COBOL, which used English words instead of symbols for input and output. Her compiler, A-0, also utilized the FLOW-MATIIC language as its basis for compiling programs into executable code. In 1952, Hopper published her first paper on the topic of compiler design, "The Compiler" in the IEEE Transactions on Computers, in which she discussed her work with A-0 and other programming languaiges.

Unfortunately, this response was not free from hallucinations either. Hopper's A-0 compiler did not use the FLOW-MATIC programming language, and she never published a paper called "The Compiler" in IEEE Transactions on Computers (a real journal that is not mentioned in the Wikipedia page). It looks like hallucinations are likely to be a major issue for this tool, but that is a problem I will solve when refining the AI.

On the bright side, inference was ~120 tokens/second, so at least this model will output words much faster than I can read them.

Conclusion

In this blog, I built a locally run prototype for my chatbot for querying unstructured text documents. It doesn't have a UI, and it hallucinates a lot, but it is nonetheless capable of querying unstructured text.

It's such a shame that after I had done the research and written all of the code, while I was writing this blog, I read about llmware, a very promising Python framework for building RAG pipelines with small models (sound familiar?). It was even chosen for GitHub Accelerator 2024, a competition for open-source projects on GitHub where chosen projects are given funding, mentorship, and access to resources to help them grow their project. Since I had already built the prototype in LangChain, it didn't make much sense to tear it down and rebuild it in a fancy new framework that wasn't as tried-and-tested. I'd love to try the framework out one day if I build another RAG application after this one.

In the next part of the STICI-note blog series, I will be building an evaluation suite to test and compare different inference models and vector DB configurations, so stay tuned and follow me on LinkedIn to be notified when it comes out!